

Inhabited Parsing

A certified recursive-descent parser for JSON

Synthesized from a *declarative grammar*, verified in **Rocq** (Coq)

`theories/Json.v` · S5 — the "control" grammar

The core idea

The **innovation**: the *implementation* and the *certification proof* can both be produced by **AI** — the **BNF is all one needs to understand**.

grammar (declarative) $\Rightarrow$ parser (executable) $\Rightarrow$ proof (certificate)

JSON grammar — high-level BNF

```
json-text ::= ws value ws

value ::= "null" | "true" | "false"
      | string | number | array | object

object ::= "{" ws members ws "}"
members ::= ε | member ws ( "," ws member ws )*
member  ::= string ws ":" ws value

array  ::= "[" ws elements ws "]"
elements ::= ε | value ws ( "," ws value ws )*

string ::= "'" char* "'"      -- no escapes in this slice
number ::= [ "-" ] int       -- no leading zeros (RFC 8259)
int     ::= "0" | digit1-9 digit*
ws      ::= ( space | tab | lf | cr )*
```

Only `value` recurses — object/array nest values through the named nonterminal `NT_value`.

The pipeline



Types at a glance — grammar & meaning

```
Spec  $\Gamma$  N A
Grammar  $\Gamma$  N := forall A, N A -> Spec  $\Gamma$  N A
denote G S  $\gamma$  i a  $\gamma'$  j
```

(* grammar: a combinator tree, as data *)
(* productions table; recursion via Call *)
(* "S, at i in state γ , yields a at j in γ' " *)

Spec parameter	what it is	in JSON
Γ	environment / state	unit (stateless)
N	nonterminal <i>family</i> N : Type -> Type	NT_value : json_nt Json
A	result type produced	Json

``denote``'s seven slots: S (spec) · γ i start (state, position) · a result · γ' j end (state, position). It is **`Type`-valued** — a derivation *is* the certificate: the parsed value ``a`` is an index of the proof.

Types at a glance — parser & data

```
parse_value (fuel : nat) (w : list ascii) : option (Json * list ascii)
parse_json  (w : list ascii) : option Json
```

```
Inductive Json : Type :=
| JNull | JBool (bool) | JNumber (Z) | JString (list ascii)
| JArray (list Json) | JObject (list (list ascii * Json)).
```

slot

reading

`fuel : nat`

fuel bound — every recursive call decrements it (termination)

`list ascii`

input as a linked list of chars (indices are plain `nat`)

`option (... × rest)`

`None` = reject; `Some (value, rest)` = value + unconsumed suffix

Supporting proof types: `Progress` (termination evidence for `Many``), `iffT A B := (A → B) × (B → A)` (Type-level iff for the ``denote_*_iff`` laws), `{ pre & pre ++ rest = w }` ("consumed a prefix").

Step 1 — grammar as data

The `Spec` deep embedding over `ascii` tokens; recursion via `Call` + a `Grammar`.

```
Inductive Spec (Γ : Type) (N : Type -> Type) : Type -> Type :=
| Pure   : forall A, A -> Spec Γ N A
| Tok    : (Token -> Type) -> Spec Γ N Token
| Seq    : Spec Γ N A -> Spec Γ N B -> Spec Γ N (A * B)
| Alt    : Spec Γ N A -> Spec Γ N A -> Spec Γ N A
| Map    : (A -> B) -> Spec Γ N A -> Spec Γ N B
| Bind   : Spec Γ N A -> (A -> Spec Γ N B) -> Spec Γ N B
| Many  : Spec Γ N A -> Spec Γ N (list A)
| Call   : N A -> Spec Γ N A.    (* recursion *)
```

```
Definition Grammar (Γ : Type) (N : Type -> Type) :=
  forall A, N A -> Spec Γ N A.
```

Semantic actions live in `Map` / `Bind` — a derivation is a *derivation-indexed JSON value*.

Step 1 — `value_spec` concretely

```
Definition value_spec : Spec ascii unit json_nt Jjson :=
  Alt (Map (fun _ => JNull) (lit "null"))
    (Alt (Alt (Map (fun b => JBool b)
      (Alt (Map (fun _ => true) (lit "true"))
        (Map (fun _ => false) (lit "false")))))
      (Map JString string_spec))
      (Alt (Map JNumber number_spec)
        (Alt (Map JArray array_spec)
          (Map JObject object_spec)))).
```

```
json_grammar NT_value = value_spec
json_text = ws · Call NT_value · ws
```


Step 2 — the denotation

`denote` is an **inductive relation** — a derivation *is* the certificate.

```
Inductive denote : forall A, Spec  $\Gamma$  N A ->  $\Gamma$  -> nat -> A ->  $\Gamma$  -> nat -> Type :=
| d_pure : ... (* Pure a: nothing consumed *)
| d_tok : nth_error w i = Some t -> P t -> ... (* one token *)
| d_seq | d_alt_l | d_alt_r | d_map | d_bind
| d_many_nil | d_many_cons
| d_call : denote (G A n)  $\gamma$  i a  $\gamma'$  j -> denote (Call n)  $\gamma$  i a  $\gamma'$  j.

(* denote G S  $\gamma$  i a  $\gamma'$  j := "S, begun at i in env  $\gamma$ ,
   produces a and ends at j in env  $\gamma'$ " - proof-relevant (Type) *)
```

Because it is `Type`-valued, ``Call`` unfolds through the grammar with no termination obligation at the spec level.

Step 3 — the parser

A fuel-bounded **mutual** Fixpoint ; first-character dispatch, ~8 levels deep.

```
Fixpoint parse_value (fuel : nat) (w : list ascii) : option (Json * list ascii) :=
  match fuel, w with
  | 0, _ | _, [] => None
  | S fuel', c :: rest =>
    if c = "{" then (* object: parse_members, expect "}" *) ...
    else if c = "[" then (* array: parse_elements, expect "]" *) ...
    else if c = '"' then (* string *) ...
    else if c = 't' / 'f' / 'n' then (* true / false / null *) ...
    else if digit / '-' then (* number *) ...
    else None
  end
(* mutual with parse_object · parse_array ·
   parse_members · parse_members_more ·
   parse_elements · parse_elements_more *)
```

``skip_ws`` runs before/after tokens; indices are plain ``nat`` cursors.

Step 4 — soundness ✓

The parser only ever produces real denotations.

```
Lemma parse_all_sound : forall fuel, (* 7-way mutual, by induction on fuel *)  
  (forall prefix w v rest, parse_value fuel w = Some (v, rest) ->  
    denote json_grammar (prefix ++ w) value_spec tt (length prefix)  
      v tt (length prefix + length w - length rest)) * ... .
```

```
Lemma parse_json_sound (w : list ascii) (v : Json) :  
  parse_json w = Some v ->  
  denote json_grammar w json_text tt 0 v tt (length w).
```

Status: all **Qed.** — value/object/array/members/members_more/elements/elements_more + top level.

Step 5 — completeness (the subtlety)

The converse is **not** a free mirror — the grammar is more permissive than the *predictive* parser.

```
Lemma parse_json_complete (w : list ascii) (v : Json) :  
  denote json_grammar w json_text tt 0 v tt (length w) ->  
  parse_json w = Some v.  (* true at the top level *)
```

- `members ::= ε / elements ::= ε` admit `Pure nil` for **any** input, but `parse_members / parse_elements` return `Some ([], w)` only when `w` starts with `}` / `]`.
- Counterexample: `denote "x" members_spec ... nil ...` holds, yet `parse_members _ "x" = None`.
- Fix: **context-aware** completeness — fold members/elements into object/array (whose spec carries the `}` / `]`), plus a ws-absorption lemma.

Step 6 — extraction & acceptance

Extraction Language OCaml.

Extraction "json.ml" parse_json. (* 649-line, self-contained *)

```
parse_json "null"      = Some JNull
parse_json "true"      = Some (JBool true)
parse_json "[1,2,3]"   = Some (JArray [JNumber 1; JNumber 2; JNumber 3])
parse_json "[true, null]" = Some (JArray [JBool true; JNull])
parse_json { "a" : 1 } = Some (JObject [("a", JNumber 1)])
```

Build: `nix develop --command make` (flake pins Rocq 9.1.1). `Eval compute` gates run per stage.

Benchmark — the inputs

- `canada.json` — 2.25 MB, from `nativejson-benchmark` : a GeoJSON `FeatureCollection` (~60k features) with floating-point coordinates. → **correctly rejected** in 0.02 s — floats sit outside the integer-only `number ::= [-] int` grammar (the parser is *sound*).
- `large_int.json` — synthetic array of objects `{"id", "name", "tags", "child", "flags"}` (integers, booleans, null, nested arrays), scaled from **4.75 MB / 30k objects** up to **53 MB / 316k objects**. → **parsed end-to-end** as `array[N]`.

Performance — extraction directives

```
Extract Inductive nat    => Z.t      (* Big_int_Z.big_int = Zarith Z.t *)
Extract Inductive Z      => Z.t
Extract Inductive ascii => char
Extract Constant Z.of_nat => "(fun n -> n)"  (* else it recurses on the value *)
```

extraction (4.75 MB)

time

naive (unary `nat` + binary `z` + `Ascii` records)

164.27 s

native `int` + `char` (63-bit)

0.137 s

`Zarith Z.t` + `char` (arbitrary precision)

0.155 s

≈ 1060× faster than the naive extraction; arbitrary precision costs only ~13% over native `int`.

Performance — scaling (Zarith)

large_int.json	size	objects	time	throughput
	4.75 MB	30k	0.155 s	~31 MB/s
	10.1 MB	63k	0.350 s	~29 MB/s
	25.9 MB	158k	1.01 s	~26 MB/s
	52.9 MB	316k	2.74 s	~19 MB/s

Thanks

Soundness certified · **Extraction** to OCaml · **Completeness** precisely scoped

```
./slides/json — npm install && npm run dev
```